

Inigo Quilez :: articles :: tracing in tiles - 2008



Intro

Imagine you are writing a software renderer based on a per pixel computation model (like when you render a **fractal** or a raytraced image). You obviously want to take advantage of the multicore architecture on your machine, and get all the threads to contribute to the rendering of the image. The easiest way to get the job done is to split the screen in tiles and have the cores work in different tiles in parallel. If you have never done that before, this is the article for you.

How NOT to render

So, you have your render algorithm set up such that you can compute the color of a pixel from the pixel coordinates. In the most simplistic case, you have something like this:

```
vec3 calcPixelColor( ivec2 pixel, ivec2 resolution );
```

Now you do have a function that needs to generate an image (a buffer of pixels) based on this function. The easiest and worst way to it would be to iterate all the pixels and render the image:

```
//-----
// render image
//-----
void renderImage( vec3 *image, ivec2 resolution )
{
    // for every pixel in this image, compute color
    for( int j=0; j<resolution.y; j++ )
        for( int i=0; i<resolution.x; i++ )
            image[xres*j+i] = calcPixelColor( ivec2(i, j), resolution );
}
```

The problem with this code is that the order in which the pixels are computed is most probably not the optimal. There are two reasons for this.

First - your calcPixelColor() is probably accessing memory in order to produce the color. From textures to ray acceleration structures, all the pixels will be requiring access to main memory. Due to the pixels being computed in raster order, all that data and memory will be read for a pixel in a given column in the current line, and will be brought to the memory cache in the hardware. As the following pixels in the scanline require new data, the cache will be keep being filled until it no longer can hold any more data, at which moment the some data in the cache is flushed in order to make room for new incoming data. When the next scanline starts and the pixel immediately below our original pixel gets rendered, the data that the former pixel brought to the cache will be gone already, meaning the data will have to be read again from main memory. This can be a problem when calcPixelColor() is using memory intensive algorithms.

Secondly, even if you weren't reading any memory in order to compute its output, still there would probably be a problem with the current raster order. Chances are that calcPixelColor() is parallelized with some **SEE** or some sort of SIMD code (in which case the look above would be incrementing *i* in bigger steps than 1). In that case, all the pixels that get rendered in parallel will probably take different branches along the code in calcPixelColor(), meaning that probably some of the pixels will have to "wait" for other pixels to finish (in practice that means that all of your "early quit" tests of the form `_mm_movemask_ps( co ) == 0x0f` will evaluate to false), which again can be computed if we had a more physically coherent way to traverse our pixels.

How to render

In order to get a more coherent memory access and compute patterns, tile rendering can be used. The idea is simple: split the screen in tiles, and render one tile at a time. The tile size should be chosen such that the data required for rendering that tile fits in the caches. In practice, it means that

you have to play a little bit with the size until you get the optimal number of pixels per tile. Usually a tile size of 8x8 to 64x64 will do quite well.

The code isn't any complex, but it can bring huge performance gains in memory heavy algorithms like [real raytracing](#). Self explanatory code:

```
//-----
// render image
//-----
void renderImage( vec3 *image, ivec2 resolution )
{
    #define TILESIZE 16

    // prep data
    const int numxtiles = resolution.x/TILESIZE;
    const int numytiles = resolution.y/TILESIZE;
    const int numtiles = numxtiles*numytiles;

    // render tiles
    for( int tile=0; i<numtiles; i++ )
    {
        // tile offset
        const int ia = TILESIZE*(tile % numxtiles);
        const int ja = TILESIZE*(tile / numxtiles);

        // for every pixel in this tile, compute color
        for( int j=0; j<TILESIZE; j++ )
        for( int i=0; i<TILESIZE; i++ )
            image[xres*(ja+j)+(ia+i)] = calcPixelColor( ivec2(ia+i, ja+j), resolution );
    }
}
```

How to render FAST

The next step is obvious: take our tiles and assign them to the different cores in our CPUs such that they can work in parallel in the image. The trick is the scheduling of which tile gets rendered by which CPU. An easy solution is to have a tile counter that indicates which is the next tile to need to be computed. When a core reads this counter and takes ownership of that tile, the counter is incremented so that next cores that want to process a new tile know which one to go for. Of course, the reading and incrementing of this counter has to be atomic in order to avoid two cores going for the same tile. The `InterlockedIncrement()` allows you to do exactly that: read a variable and increment it atomically. If the tile counter reaches a number which is equal or larger than the amount of tiles in the image, the thread can die of course, cause it means there's no more work to be done.

The threads run will run the `doRender()` in parallel, and they'll access the global data they are operating in (the image buffer and the tile counter, for example) from a pointer to the local stack of the main thread (the one that actually executes `renderImage()`). In desperate cases of size optimization this could be done through global variables, but of course in real case scenarios that's not an option. The data with this global information is packed in the `TData` structure.

When this scheduling mechanism is in place, all that is left is to create as many threads as hardware cores the running machine has (which the operative system makes available to you through the `GetSystemInfo()` function), and let them start consuming tiles. The main thread will of course have to wait until this "working" threads are done. We can use a semaphore for that, which we will have to create prior to the creation of the worker threads, and to which the main thread will have to block by calling `WaitForMultipleObjects()`.

All of these operations don't actually need a big amount of code. In fact, the code bellow is full functional and can be simply copy and pasted in your projects, it's ready to function:

```
#include <windows.h>
#include "vecTypes.h"

vec3 calcPixelColor( ivec2 pixel, ivec2 resolution );

typedef struct
{
    long    tileID;
    HANDLE  barrier;
    vec3    *image;
    ivec2    resolution;
}TData;

static unsigned long __stdcall doRender( void *vdata )
{
    #define TILESIZE 16

    // prep data
    TData *data = (TData*)vdata;
    const ivec2 resolution = data->resolution;
    const int numxtiles = resolution.x/TILESIZE;
    const int numytiles = resolution.y/TILESIZE;
    const int numtiles = numxtiles*numytiles;

    // synch point
```

```

//-----
// render image
//-----
void renderImage( vec3 *image, ivec2 resolution )
{
    // get number of CPUs available
    SYSTEM_INFO si;
    GetSystemInfo( &si );
    const int numCpus = min( si.dwNumberOfProcessors, 256 );

    // data for worker threads
    __declspec(align(32)) TData data = {
        0, // tile id/counter
        CreateSemaphore( 0, numCpus, numCpus, 0 ), // create barrier
        image, resolution // image
    };

    // launch worker threads
    HANDLE th[256];
    for( const int i=0; i<numCpus; i++ )
        th[i] = CreateThread( 0, 0, doRender, &data, 0, 0 );

    // wait for worker threads to finish
    WaitForMultipleObjects( numCpus, th, true, INFINITE);

    // destroy threads
    for( int i=0; i<numCpus; i++ )
        CloseHandle( th[i] );

    // destroy barrier
    CloseHandle( data.barrier );
}

WaitForSingleObject( data->barrier, 0 );

// render tiles
for(;;)
{
    // get next tile to consume
    const int tile = InterlockedIncrement( &
    if( tile ≥ numtiles ) break;

    // tile offset
    const int ia = TILESIZE*(tile % numxtile
    const int ja = TILESIZE*(tile / numxtile

    // for every pixel in this tile, compute
    for( int j=0; j<TILESIZE; j++ )
    for( int i=0; i<TILESIZE; i++ )
        data->image[xres*(ja+j)+(ia+i)] = ca
        vec2(ia+i, ja+j),
    }

    // synch point
    ReleaseSemaphore( data->barrier, 1, NULL );
    return 1;
}

```

You also have a smaller version of the code above in the [1k and 4k demo framework](#) available in this website (which I used for many demoscene productions from 2006 to 2009).

inigo quilez - learning computer graphics since 1994